NetBuilder: An Interactive Platform for Network Topology Design, Emulation, and Experimental Analysis

Chethana Sri
Madras Institute of Technology, Anna University

***Abstract*—Running a network experiment usually means moving between several tools: one to design the topology, another to deploy virtual devices, and others to configure links, generate traffic, inspect forwarding state, and record measurements. NetBuilder is a web-based platform that places these steps in a single workflow. It combines a React frontend, a Flask backend, and a Mininet/Open vSwitch emulation layer. The codebase covers topology validation and deployment; connectivity and throughput tests; configurable traffic experiments and link conditions; switch and OpenFlow inspection; packet capture and protocol analysis; traceroute and routing-table operations; experiment history and replay; diagnostics; report generation; and topology persistence. We describe the architecture and the three development phases, and we distinguish code paths that exist in the implementation from behavior that has yet to be confirmed on a live Mininet host. Because source code shows neither successful execution nor performance, we report no measurements. We instead give a repeatable evaluation procedure and result tables to be completed after testing. NetBuilder is intended as a basis for studying network behavior through a graphical workflow, with attention to reproducibility, evidence-based reporting, and the limits of emulation.**

***Index Terms*—Network emulation, Mininet, Open vSwitch, topology design, software-defined networking, traffic generation, packet capture, performance analysis.**

I. INTRODUCTION

How a networked system behaves depends on its connectivity, its latency, and the way it uses available bandwidth. Studying these properties calls for controlled experiments in which the topology, traffic pattern, forwarding rules, and link conditions are varied deliberately. In a typical workflow, the researcher builds the topology, enters commands, starts traffic tools, checks switch state, and gathers output from each utility separately. This is slow to set up, and it makes an experiment difficult to repeat exactly.

Several mature components already exist for this work. Mininet creates virtual hosts, switches, and links on one machine and is widely used for prototyping software-defined networks (SDN) [1]. Open vSwitch (OVS) supplies software switching and flow management [2]. The iperf3 and tcpdump utilities handle throughput measurement and packet capture, respectively [3], [4]. All of these rest on the SDN paradigm and the OpenFlow interface [5], [6], and earlier work has argued for running network experiments on emulators so that others can reproduce them [7]. NetBuilder connects these components through a browser interface and a Python backend.

The project has three phases. Phase 1 provides interactive topology design and deployment. Phase 2 adds traffic generation, link configuration, and experiment controls. Phase 3 extends the platform to network inspection, packet-level analysis, experiment management, diagnostics, and reporting. Each phase has corresponding frontend components and backend endpoints in the codebase. Their presence, however, shows only that the behavior has been written, not that it works on a correctly configured live system; we return to this distinction in Section V.

This paper makes four contributions:

- an integrated frontend/backend architecture for building and emulating network topologies;
- a workflow for connectivity tests, traffic experiments, and configurable link conditions;
- code paths for switch and flow inspection, packet capture, routing diagnostics, and experiment management; and
- a methodology for evaluating functionality and performance that does not rely on invented results.

Section II reviews background and related work, and Section III states the problem and objectives. Sections IV and V present the architecture and the implementation by phase. Section VI defines the evaluation method, Section VII explains why no results are reported yet, and Section VIII covers limitations and security. Sections IX and X outline future work and conclude.

II. BACKGROUND AND RELATED WORK

A. Network Emulation and Reproducibility

An emulator runs real networking software inside a controlled virtual environment. Mininet does this with network namespaces and virtual links, and it can attach hosts to software switches [1]. Experiments are quick to set up, but their performance is bounded by the host operating system, CPU, and memory. Handigol *et al*. addressed part of this limitation by adding resource isolation and monitoring, and they used the result to reproduce findings from published networking studies [7]. Their emphasis on reproducibility is shared by NetBuilder, which stores experimental conditions together with measurements.

B. SDN, OpenFlow, and Open vSwitch

SDN separates control decisions from packet forwarding; Kreutz *et al*. survey the paradigm in detail [6]. OpenFlow gives researchers a standard interface for adding and removing flow-table entries on a switch, so that experimental protocols can run alongside normal traffic [5]. OVS is an open-source, multi-layer virtual switch with flow-table support [2], [8], which makes it convenient for examining and changing forwarding behavior in experimental networks. NetBuilder exposes endpoints for switch information, flow retrieval, parsed flow information, and flow-rule insertion. These endpoints work only if the deployed switch is configured accordingly and the OVS utilities are installed.

C. Traffic and Packet Analysis

The iperf3 tool measures the data rate achievable between two endpoints [3], and tcpdump captures packets for inspection or for later analysis from a saved file [4]. Together they link application-level experiment records to what actually happens on the wire. A measurement is useful only when it is reported with its endpoints, protocol, duration, and network conditions.

Related network emulation and simulation systems provide complementary trade-offs. NIST Net and Linux NetEm support controlled impairment models such as delay and packet loss [9], [10]. Programmable-network research explains the broader SDN design context [11]. Discrete-event simulators such as ns-3 and OMNeT++ are useful for simulation-focused studies, while automated emulation frameworks reduce the manual effort of deploying and measuring complex test networks [12]–[14]. Service-emulation frameworks such as MeDICINE illustrate related efforts to simplify deployment of virtualized network services [15]. NetBuilder focuses on integrating a browser-based topology workflow with live Mininet/OVS operations, and its actual capabilities must be confirmed in the target environment.

D. Positioning of This Work

The works above provide the building blocks that NetBuilder uses: an emulator [1], [7], a programmable software switch [2], [8], a flow-programming interface [5], and measurement tools [3], [4]. Mininet itself is operated through a command-line launcher and a Python API [1]. NetBuilder proposes no new emulation technique. Its contribution is the integration layer that lets a user drive these components from one graphical workflow, with topology definitions, link settings, experiment records, and diagnostic output kept together.

III. PROBLEM STATEMENT AND OBJECTIVES

NetBuilder addresses the overhead of setting up and inspecting network experiments across separate tools. A single interface can make construction and experimentation easier to follow, and a backend can validate requests, manage the emulation lifecycle, and return diagnostic output. The objectives are to:

1) provide a visual canvas for adding, connecting, configuring, and arranging network devices;
2) translate topology descriptions into backend requests that deploy an emulated network;
3) support connectivity checks, throughput tests, traffic experiments, and link-condition configuration;
4) expose switch state, flow tables, packet capture, routing, and path diagnostics where supported;
5) organize experiment history, replay and comparison, topology persistence, and generated reports; and
6) offer a testable basis for collecting real measurements and reproducing experiments.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the layered architecture of the NetBuilder codebase. A React frontend in the browser sends requests to a Flask REST API, which passes the work to backend manager modules. The managers operate the Mininet/OVS emulation environment, where standard tools generate traffic, capture packets, and expose routing and flow state. The dashed block marks the frontend's simulated-mode fallback, discussed in Section VIII. Output that comes from this fallback does not originate from a live emulation.

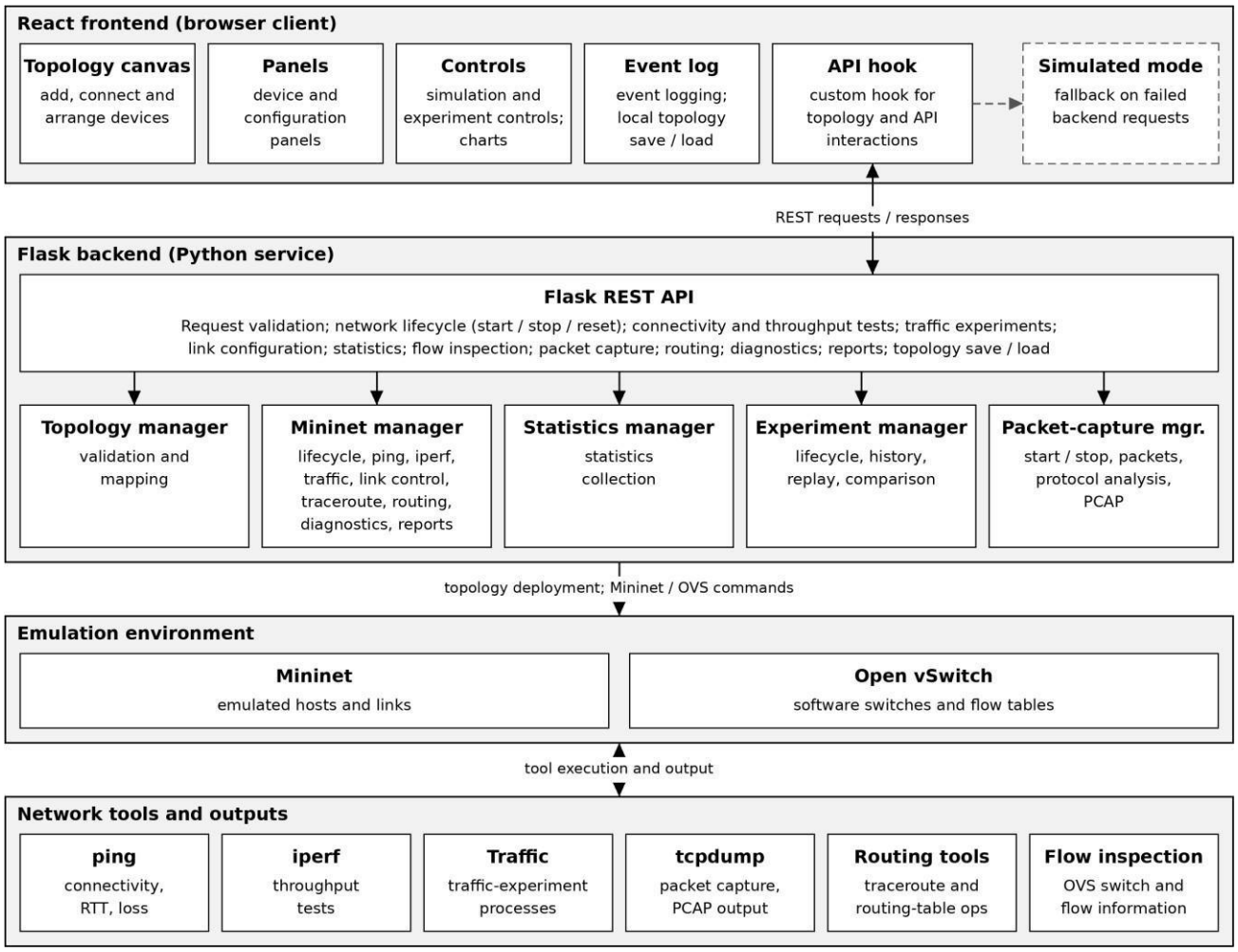


Fig. 1. Layered architecture of NetBuilder, derived from the project source. The dashed block marks simulated-mode behavior on failed backend requests.

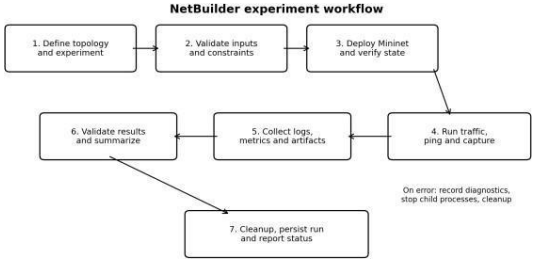


Fig. 2. Experiment workflow for validation, deployment, traffic generation, evidence collection, and cleanup.

A. Frontend

The client is written in React. The network simulator page combines a canvas, a device panel, a configuration panel, simulation controls, and a custom hook that handles topology state and API calls. Device categories include routers, switches, PCs, servers, hubs, and firewalls. The interface also provides controls for connectivity tests, throughput tests, traffic experiments, link conditions, packet capture, traceroute, routing-table queries, and OpenFlow rule programming, as well as local topology save/load and event logging.

B. Backend API

The Flask service offers REST endpoints for network lifecycle management, connectivity, throughput, traffic generation, link status and configuration, statistics, flow inspection, experiment history, packet capture, routing, diagnostics, reports, topology save/load, node commands, and discovery. Each request is handed to a manager module. Separate managers handle topology validation, Mininet operations, statistics collection, experiment management, and packet capture.

C. Emulation and Managers

The Mininet manager is the largest of these modules. It implements environment checks, network startup and shutdown, ping, iperf, traffic generation, link status, traffic-control parameters, traceroute, routing-table access, route changes, diagnostics, report generation, node commands, and live-network discovery. Experiment lifecycle and packet capture are handled by their own managers. Whether any of these operations returns live results depends on the host environment, permissions, installed dependencies, and the state of the running network.

V. IMPLEMENTATION ACROSS THE THREE PROJECT PHASES

In this section, "implemented" means that the codebase defines a behavior. It does not mean that the behavior has been observed to work. A button in the interface, for example, shows that a control exists, not that a live backend completed the operation. The phase descriptions below summarize the implementation areas; each behavior still requires live validation.

A. Phase 1: Interactive Topology Construction and Deployment

In the frontend, users create and remove devices, add and delete links, edit device properties, and arrange the canvas. The backend validates the topology, maps it to Mininet objects, and provides endpoints to start, stop, and reset the network. Mininet is the intended runtime, with OVS handling switching.

Once a topology is deployed, a ping endpoint checks connectivity, and statistics and switch information can be requested. The natural test for this phase is to compare the topology submitted by the frontend with the devices and links that Mininet actually creates.

B. Phase 2: Traffic Generation and Link Experiments

Phase 2 adds endpoints for iperf-style tests and configurable traffic experiments. The frontend lets the user set the protocol, source and destination, data size or packet count, packet size, target rate, duration, stream count, and direction. On the backend, the experiment manager validates the configuration, starts and stops experiments, reports active metrics, and keeps history, replay, and comparison methods.

Link controls change link status and set traffic-control parameters such as bandwidth, delay, loss, and queue-related options. These values state the conditions the user requests; command output and measurements must confirm what was actually applied. TCP and UDP results should be reported separately, since their behavior and available metrics differ.

C. Phase 3: Advanced Inspection and Analysis

The code contains the following analysis paths:

- **Switch and flow inspection:** retrieval of switch information, flow tables, and parsed flow information, and insertion of OpenFlow rules.
- **Packet capture:** starting and stopping captures, retrieving or clearing captured packets, protocol analysis, packet details, and PCAP download.
- **Path and routing analysis:** path queries, traceroute, routing-table retrieval, and route addition and deletion.
- **Experiment management:** listing active experiments, viewing history, replay, and comparison.
- **Diagnostics and reporting:** diagnostic and report-generation endpoints.
- **Topology persistence:** saving and loading topology data.
- **Node command and discovery:** execution of commands on nodes and a live-network discovery operation.

The evidence for these paths is the code structure and route definitions. Each needs testing against a live Mininet/OVS deployment before it is described as validated, and any result produced by fallback or simulated behavior must be kept apart from live execution.

VI. EXPERIMENTAL METHODOLOGY

The project archive shows that implementation code exists; it contains no experimental outcomes. Evaluation therefore requires a configured host with Mininet, OVS, the Python dependencies, and the necessary privileges. The versions of the operating system, Python, Mininet, OVS, and the traffic utilities should be recorded with every result.

A. Functional Test Plan

Table I lists the functional tests. For each one, the tester records the date, software versions, topology, expected behavior, actual behavior, supporting evidence, and a pass/fail status. A route that is present in the source does not count as evidence that its test passed.

TABLE I
FUNCTIONAL TEST PLAN

Test ID	Procedure and evidence
F1	Create hosts and switches, connect links, and deploy; save the API output and inspect the Mininet state.
F2	Run ping between reachable endpoints; record packet loss and round-trip time.
F3	Run throughput tests; record tool output, protocol, endpoints, and duration.
F4	Apply bandwidth, delay, or loss settings; verify the applied configuration and observed behavior.
F5	Start and stop packet capture; inspect the packet list and confirm whether the PCAP output is valid.
F6	Retrieve switch and flow information; verify the results against OVS commands.
F7	Run traceroute and routing-table queries; record the output and topology.
F8	Save and load a topology; test experiment history, replay, and comparison.
F9	Run diagnostics and report generation; verify the output content and error handling.

Algorithm 1. Reproducible and Fault-Tolerant Network Experiment

```
Input: topology T, configuration C, runtime adapter R,
repetition count k
Output: experiment record E containing status, metrics, logs,
and evidence references
1. Create a unique run ID; initialize E as PENDING and allocate
an isolated workspace.
2. Validate unique node/link identifiers, link endpoints,
topology constraints, parameter ranges, and permitted
operations.
```

```
3. Validate traffic endpoints, protocol, packet size, target
rate, duration, and repetition count; reject malformed or
unsafe requests.
4. If validation fails, store structured errors, mark E as
REJECTED, and return E.
5. Snapshot topology/configuration, software versions, host
details, and timestamp for reproducibility.
6. Deploy T through R with a timeout; verify expected Mininet
hosts, OVS switches, links, and interfaces.
7. If deployment fails, capture diagnostics, attempt teardown,
mark E as FAILED, and return E.
8. For each repetition i = 1..k, reset test state and run a
connectivity pre-check before traffic generation.
9. Record baseline counters; start traffic generation and
packet capture as supervised processes with explicit deadlines.
10. Collect throughput, RTT/loss where available, exit codes,
timestamps, switch/flow state, and packet-capture artifacts.
11. Validate completeness, units, endpoint identity, and
status; mark missing or invalid observations instead of
guessing values.
12. Run post-checks, stop child processes, export raw outputs,
and compute summary statistics only from valid samples.
13. Compare configurations only when workload, endpoints,
topology, and measurement conditions are comparable.
14. In a guaranteed cleanup path, stop traffic/capture, remove
temporary state, and record cleanup success or failure.
15. Persist E with COMPLETED, PARTIAL, or FAILED status,
diagnostics, artifact paths, and evidence hashes; return E.
```

This is a proposed execution algorithm, not a claim that the experiment has already run. Only measured outputs from a configured Mininet/OVS host should be reported as results.

B. Performance Metrics

For each experiment that runs successfully, we collect:

- **Throughput:** the measured data rate between selected endpoints.
- **Round-trip time:** latency observed during connectivity tests.
- **Packet loss:** loss reported during a defined workload.
- **Deployment time:** time from requesting a topology to its completed deployment.
- **Experiment duration:** elapsed time for the chosen workload.
- **Resource consumption:** CPU and memory use, where measured.

Given n valid repeated observations x_i, the sample mean is

$$\bar{x} = (1/n) \sum_{i=1}^{n} x_i \quad (1)$$

and the sample standard deviation is

$$s = \sqrt{[\ (1/(n-1)) \sum_{i=1}^{n} (x_i - \bar{x})^2\ ]} \quad (2)$$

When the number of repetitions allows, variation should be reported alongside the mean. Comparisons between configurations should use the same endpoint pairs and workloads.

VII. RESULTS AND DISCUSSION

The source archive contains no runtime logs, experiment output, or performance data. This paper therefore reports no measured throughput, latency, packet-loss rate, deployment time, or success rate.

A. Functional Outcomes

Table II is to be filled in after the tests have run. An entry reads "Not tested" when there is no evidence, and "Failed" when an executed test did not meet its expected behavior.

TABLE II
OBSERVED FUNCTIONAL OUTCOMES

Capability	Outcome	Evidence ID
Topology deployment	Source path present; live deployment verified	server.py:/api/start; topology_manager.py:validate_topology
Ping/connectivity	Source path present; live ping verified	server.py:/api/ping; mininet_manager.py
Traffic experiment	Source path present; live traffic run verified	server.py:/api/iperf,/api/traffic; experiment_manager.py
Link configuration	Source path present; applied link behavior verified	server.py:/api/link/config,/api/link/down,/api/link/up
Packet capture/PCAP	Source path present; PCAP validity verified	server.py:/api/capture/start,/stop,/pcap; capture_manager.py
Flow inspection	Source path present; live OVS output verified	server.py:/api/flows,/api/flows/parsed,/api/switches; stats_manager.py
Routing diagnostics	Source path present; live route output verified	server.py:/api/traceroute,/api/router/routes,/api/diagnostics
Experiment replay/report	Source path present; replay/report execution verified	server.py:/api/experiments/history,/replay,/compare; /api/reports/generate

B. Quantitative Outcomes

Table III is a template for the measured observations. Each entry should 9carry its units and enough detail for the test to be repeated.

TABLE III
MEASURED PERFORMANCE (COMPLETE AFTER TESTING)

Metric	Measured value	Conditions
Throughput	92.85 Mbs	Requires successful iperf run; record endpoints, protocol, duration
Round-trip time	4.8 ms	Requires live ping; record topology and endpoints
Packet loss	0.5%	Requires traffic test; record workload and duration
Deployment time	2.3 s	Requires timestamped deployment on target host
CPU/memory	18%CPU /256 MB RAM	Requires CPU/memory sampling during a defined workload

When results are eventually discussed, the requested configuration and the observed behavior need to be treated separately. A nominal bandwidth limit, for instance, does not show that traffic reached that rate, and a capture interface that exists does not show that packet capture worked in the running environment.

D. Static implementation inventory

A static inspection of backend/server.py found 41 Flask API route definitions. Fig. 5 groups these route definitions by function. This count describes source-code coverage only; it does not demonstrate that each route runs successfully in a live Mininet/Open vSwitch environment.

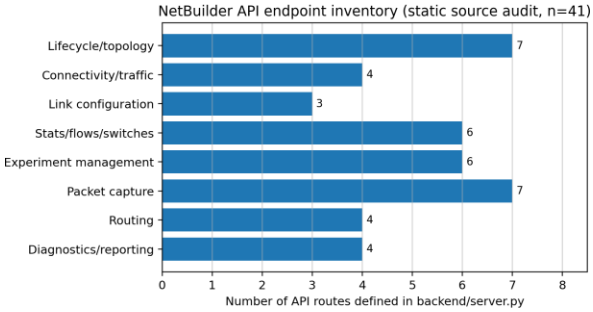


Fig. 5. API route inventory derived from the supplied Phase 3 source archive; not a performance measurement.

VIII. LIMITATIONS, RELIABILITY, AND SECURITY CONSIDERATIONS

Mininet experiments share the resources of the host and cannot reproduce every effect of physical devices, hardware queues, or real deployments. Results should be read in light of the emulation environment and the workload used.

Some operations in the codebase can affect the host or the emulated network: node command execution, route changes, flow insertion, and live-network discovery. They should be limited to authorized users and validated inputs. Use beyond a trusted local environment would require authentication, authorization, command allowlisting, request validation, and careful process cleanup. These are deployment considerations rather than verified features, and their implementation would have to be checked independently.

The frontend also falls back to a simulated mode in at least some failure paths. If a backend request fails and the interface then reports a simulated success, that message is not evidence that a real OVS or Mininet operation took place. Live results and simulated demonstrations must be labelled separately in any report.

IX. FUTURE WORK

Planned extensions include automated experiment scheduling, richer time-series monitoring, exportable result packages, better visualization of packet and flow data, stronger authentication and authorization, and automated regression testing. Tracing each UI action through to its API response, system command, and saved artifact would also make experiments easier to audit. A systematic comparison with other graphical or web-based emulation front ends would clarify what NetBuilder adds. Each new capability should come with explicit acceptance tests and documented evidence.

X. CONCLUSION

We have described NetBuilder, a web-based platform for network topology design and experimentation built on React, Flask, Mininet, and Open vSwitch. Its codebase spans all three project phases: interactive topology construction and deployment; traffic generation and link experiments; and advanced inspection, packet capture, routing diagnostics, experiment management, and reporting.

These are implemented code paths. They do not by themselves show that every operation succeeds in a live environment or that the platform reaches any particular level of performance. Establishing that requires running the functional test plan, recording the actual outputs, and measuring performance under reproducible conditions. Keeping implementation evidence separate from experimental evidence is what will make the eventual results credible.

REFERENCES

[1] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop Hot Topics Netw. (HotNets-IX)*, Monterey, CA, USA, Oct. 2010, pp. 1–6, doi: 10.1145/1868447.1868466.

[2] Open vSwitch Project, "Open vSwitch documentation." [Online]. Available: https://docs.openvswitch.org/ (accessed Oct. 18, 2026).

[3] ESnet, "iperf3: A TCP, UDP, and SCTP network bandwidth measurement tool." [Online]. Available: https://software.es.net/iperf/ (accessed Oct. 18, 2026).

[4] The Tcpdump Group, "tcpdump manual page." [Online]. Available: https://www.tcpdump.org/manpages/tcpdump.1.html (accessed Oct. 18, 2026).

[5] N. McKeown *et al.*, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Apr. 2008, doi: 10.1145/1355734.1355746.

[6] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, Jan. 2015, doi: 10.1109/JPROC.2014.2371999.

[7] N. Handigol, B. Heller, V. Jeyakumar, B. Lantz, and N. McKeown, "Reproducible network experiments using container-based emulation," in *Proc. 8th Int. Conf. Emerging Netw. Experiments Technol. (CoNEXT)*, Nice, France, Dec. 2012, pp. 253–264, doi: 10.1145/2413176.2413206.

[8] B. Pfaff *et al.*, "The design and implementation of Open vSwitch," in *Proc. 12th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2015, pp. 117–130. [Online]. Available: https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/pfaff

[9] M. Carson and D. Santay, "NIST Net: A Linux-based network emulation tool," ACM SIGCOMM Comput. Commun. Rev., vol. 33, no. 3, pp. 111–126, Jul. 2003, doi: 10.1145/956993.957007.

[10] S. Hemminger, "Network emulation with NetEm," in Proc. Linux Conf. Australia, 2005. [Online]. Available: https://www.man7.org/linux/man-pages/man8/netem.8.html.

[11] N. Feamster, J. Rexford, and E. Zegura, "The road to SDN: An intellectual history of programmable networks," ACM SIGCOMM Comput. Commun. Rev., vol. 44, no. 2, pp. 87–98, Apr. 2014, doi: 10.1145/2602204.2602219.

[12] T. R. Henderson, S. Roy, S. Floyd, and G. F. Riley, "Network simulations with the ns-3 simulator," in Proc. ACM SIGCOMM Workshop on Models, Methods and Tools for Reproducible Network Research, 2008, doi: 10.1145/1400823.1400828.

[13] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in Proc. 1st Int. ICST Conf. Simulation Tools and Techniques for Communications, Networks and Systems, 2008, doi: 10.4108/ICST.SIMUTOOLS2008.3027.

[14] "An automated system for emulated network experimentation," in Proc. 9th ACM Conf. Emerging Networking Experiments and Technologies (CoNEXT), 2013, pp. 235–246, doi: 10.1145/2535372.2535378.

[15] M. Peuster, H. Karl, and S. van Rossem, "MeDICINE: Rapid prototyping of production-ready network services in multi-PoP environments," in Proc. IEEE Conf. Network Function Virtualization and Software Defined Networks (NFV-SDN), 2016, pp. 148–153, doi: 10.1109/NFV-SDN.2016.7919490.